



CS671: Deep Learning Assignment 3

Autoencoder (AE)

Group Number: 20

Course: CS671

Institution: Indian Institute of Technology Mandi

Submitted By:

Akshat Mittal (B24104)

Akshita Dangi (B24106)

Aman Gupta (B24107)

Arihant Kumar Jain (B24112)

Saksham Kundu (B24154)

Vandana Dutt (B24175)

Priansh Yajnik (B24318)

Vansh Pandey (B24411)

Submission Date: March 18, 2026

Contents

Part 1: The Bottleneck Challenge (Undercomplete AE)	3
Problem Definition	3
Architecture	3
Methodology	3
Results	4
Latent Space Visualization ($N = 2$)	4
Latent Space Visualization ($N = 32$)	5
Reconstruction with Bottleneck Size $N = 2$	5
Reconstruction with Bottleneck Size $N = 32$	6
Key Findings	6
Conclusion	6
Part 2: Denoising & Feature Robustness	6
Model Architecture	6
Encoder	6
Decoder	7
Why Convolutional Refinement Layers After Each Upsampling Step?	7
Experiments and Results	8
Experiment 1 – Standard Reconstruction Autoencoder	8
Experiment 2 – Denoising Autoencoder (DAE)	8
Reconstruction Trio Visualisation	8
Loss Function and Evaluation Metric	8
Why MSE is the Right Loss for Image Reconstruction	8
Why SSIM is a Better Evaluation Metric Than R^2	9
Conclusion	9
Part 3.1: Latent Space Visualization & Analysis (2D Manifold)	9
Introduction	9
Methodology	9
Architecture	9
Training Details	9
Results	10
Reconstruction Loss	10
Original vs Reconstructed Images	10
2D Latent Space Scatter Plot	11
Analysis	11
Part 3.2: Anomaly Detection using Convolutional Autoencoders	11
Problem Definition & Methodology	11
Hyperparameter Tuning & Architecture Design	12
Results, Visualizations, and Interpretations	13
Key Findings and Explanations	14
Part 3: Ablation Study – Effect of Output Activation Function	14
Objective	14
Model Setup	15
Results	15
Reconstruction Loss Analysis	15
Reconstruction Comparison	16
Discussion and Analysis	16

Conclusion 17

Part 1: The Bottleneck Challenge (Undercomplete AE)

Problem Definition

Autoencoders are neural networks used for unsupervised representation learning. They compress input data into a lower-dimensional representation called a **latent space** and then reconstruct the original input.

The objective of this experiment is to investigate how the size of the latent bottleneck affects reconstruction quality when compressing handwritten digit images from the MNIST dataset.

Architecture

The implemented symmetric autoencoder consists of the following structure.

Encoder

$$784 \rightarrow 128 \rightarrow 64 \rightarrow N$$

Decoder

$$N \rightarrow 64 \rightarrow 128 \rightarrow 784$$

Where N represents the latent bottleneck dimension.

Two experiments were performed:

- Bottleneck size $N = 2$
- Bottleneck size $N = 32$

Methodology

- Dataset: MNIST handwritten digits
- Input Size: 28×28 grayscale images
- Flattened Input Vector: 784
- Loss Function: Mean Squared Error (MSE)
- Optimizer: Adam
- Learning Rate: 0.001
- Epochs: 10
- Batch Size: 128

Each image was flattened into a vector of size 784 before being passed to the network.

Results

Latent Space Visualization ($N = 2$)

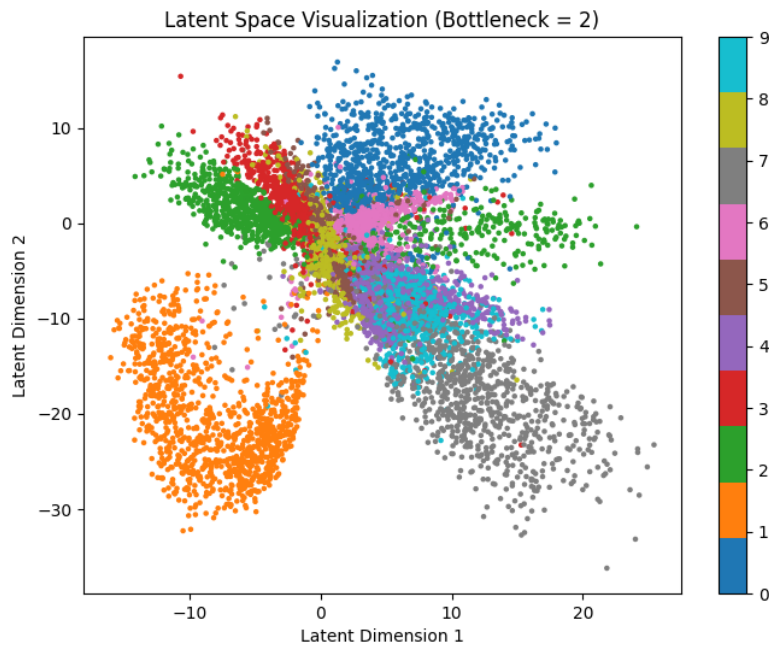


Figure 1: 2D latent representation learned by the autoencoder with bottleneck size 2.

This visualization shows how the encoder organizes digit images into a compressed two-dimensional space. Even though the model was trained without labels, similar digits tend to cluster together, indicating that the network has learned meaningful structural features of the digits.

Latent Space Visualization ($N = 32$)

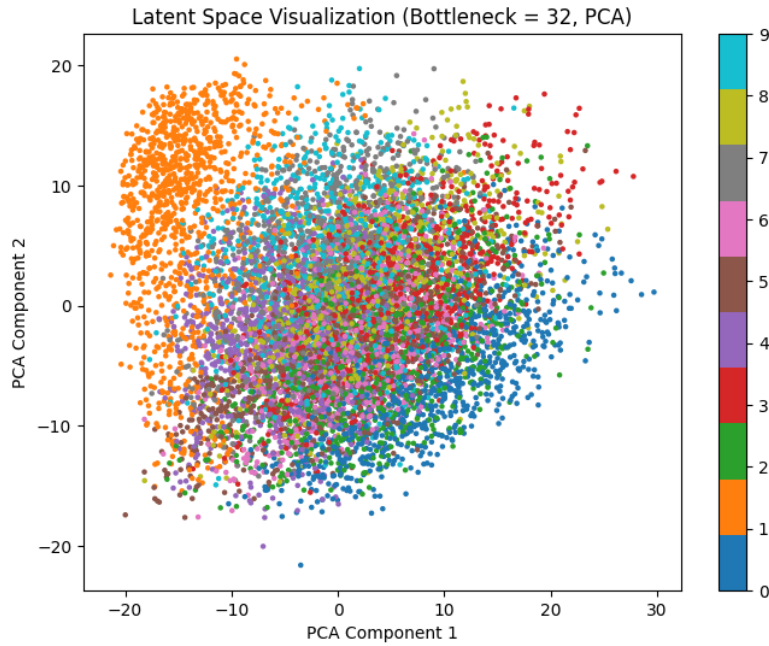


Figure 2: Latent representation for bottleneck size 32 projected to two dimensions using PCA.

Since the latent representation contains 32 dimensions, PCA was used to reduce it to two dimensions for visualization. The clusters appear more structured because the higher-dimensional latent space allows the network to encode more detailed information about each digit.

Reconstruction with Bottleneck Size $N = 2$



Figure 3: Original vs reconstructed MNIST digits using a bottleneck size of 2.

With only two latent variables, the network compresses the entire image into an extremely small representation. As a result, the reconstructed images appear blurry and lose important structural details of the digits.

Reconstruction with Bottleneck Size $N = 32$



Figure 4: Original vs reconstructed MNIST digits using a bottleneck size of 32.

With a larger latent space, the network can preserve significantly more information about the input image. This leads to clearer reconstructions and better preservation of digit structure.

Key Findings

- The latent bottleneck dimension strongly affects reconstruction quality.
- Extremely small bottlenecks force aggressive compression and cause information loss.
- Larger latent spaces allow the model to encode more meaningful features.
- Autoencoders learn compact representations while balancing compression and reconstruction accuracy.

Conclusion

This experiment demonstrates the importance of latent space dimensionality in autoencoder architectures. A very small bottleneck forces the model to learn only the most dominant features, resulting in degraded reconstructions. Increasing the bottleneck size allows the model to preserve more information, producing higher quality reconstructions.

Part 2: Denoising & Feature Robustness

Model Architecture

The model is a symmetric convolutional autoencoder implemented in PyTorch. It consists of two sub-networks: an **Encoder** that compresses the input image into a compact latent representation, and a **Decoder** that reconstructs the clean image from that representation. Both experiments share *identical* architecture; only the training inputs differ.

Encoder

The encoder maps a $3 \times 64 \times 64$ RGB image to a bottleneck tensor of shape $256 \times 8 \times 8$ through three successive $Conv2d \rightarrow BatchNorm \rightarrow LeakyReLU \rightarrow MaxPool2d$ blocks, each doubling the channel depth while halving the spatial resolution.

Block	Operation	In Ch.	Out Ch.	Kernel/Stride	Output Shape
1	Conv2d + BN + LeakyReLU(0.1)	3	64	3×3 , pad=1	$64 \times 64 \times 64$
	MaxPool2d	–	–	2×2	$64 \times 32 \times 32$
2	Conv2d + BN + LeakyReLU(0.1)	64	128	3×3 , pad=1	$128 \times 32 \times 32$
	MaxPool2d	–	–	2×2	$128 \times 16 \times 16$
3	Conv2d + BN + LeakyReLU(0.1)	128	256	3×3 , pad=1	$256 \times 16 \times 16$
	MaxPool2d	–	–	2×2	$256 \times 8 \times 8$

Table 1: Encoder architecture

LeakyReLU(0.1): Standard ReLU permanently kills neurons with negative pre-activations. A small negative slope of 0.1 allows gradients to flow for negative inputs, preventing dead neurons and producing richer latent encodings.

BatchNorm2d: Applied after each convolution to normalise feature maps per channel. This accelerates convergence, reduces internal covariate shift, and provides mild regularisation.

Decoder

The decoder inverts the spatial compression using three **ConvTranspose2d** (learnable upsampling) blocks, each immediately followed by an additional **Conv2d** refinement layer.

Block	Operation	In Ch.	Out Ch.	Kernel/Stride	Output Shape
1	ConvTranspose2d + BN + LeakyReLU	256	128	2×2 , stride=2	$128 \times 16 \times 16$
	Conv2d + LeakyReLU (refine)	128	128	3×3 , pad=1	$128 \times 16 \times 16$
2	ConvTranspose2d + BN + LeakyReLU	128	64	2×2 , stride=2	$64 \times 32 \times 32$
	Conv2d + LeakyReLU (refine)	64	64	3×3 , pad=1	$64 \times 32 \times 32$
3	ConvTranspose2d + BN + LeakyReLU	64	32	2×2 , stride=2	$32 \times 64 \times 64$
	Conv2d + Sigmoid (output)	32	3	3×3 , pad=1	$3 \times 64 \times 64$

Table 2: Decoder architecture

Why Convolutional Refinement Layers After Each Upsampling Step?

A common simplification is to use only **ConvTranspose2d** with no follow-up convolution. While this reduces parameter count, it introduces a well-documented problem — **checkerboard artefacts**.

The additional 3×3 **Conv2d** inserted *after* each **ConvTranspose2d** serves three purposes:

1. **Artefact suppression:** The convolution re-mixes neighbouring pixels, smoothing the uneven checkerboard pattern.
2. **Feature refinement:** Extra learnable parameters at each spatial scale allow the decoder to reconstruct finer textures and structural detail.

3. **Additional non-linearity:** Paired with LeakyReLU, each refinement layer increases the decoder’s expressive capacity.

The final layer uses **Sigmoid** to constrain all output pixel values to $[0, 1]$, matching the normalised range produced by `transforms.ToTensor()`.

Experiments and Results

Experiment 1 – Standard Reconstruction Autoencoder

In the first experiment, the autoencoder was trained to reconstruct its *clean* input directly: the same image served as both the network input and the reconstruction target.

Training objective:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^N \|f_{\theta}(x_i) - x_i\|^2$$

Optimiser: Adam, $lr = 10^{-2}$, 50 epochs, batch size 32.

After 50 epochs on Tiny-ImageNet-10, the model achieved a test-set SSIM of **71%**, confirming that the architecture has sufficient capacity to learn compact and recoverable image representations.

Experiment 2 – Denoising Autoencoder (DAE)

In the second experiment, Gaussian noise was injected into each image before it was passed to the encoder, while the *clean* original image remained the reconstruction target.

The DAE achieved a test-set SSIM of **70%** – only 1 percentage point below the standard autoencoder, despite the significantly harder task of recovering clean images from corrupted inputs.

Reconstruction Trio Visualisation

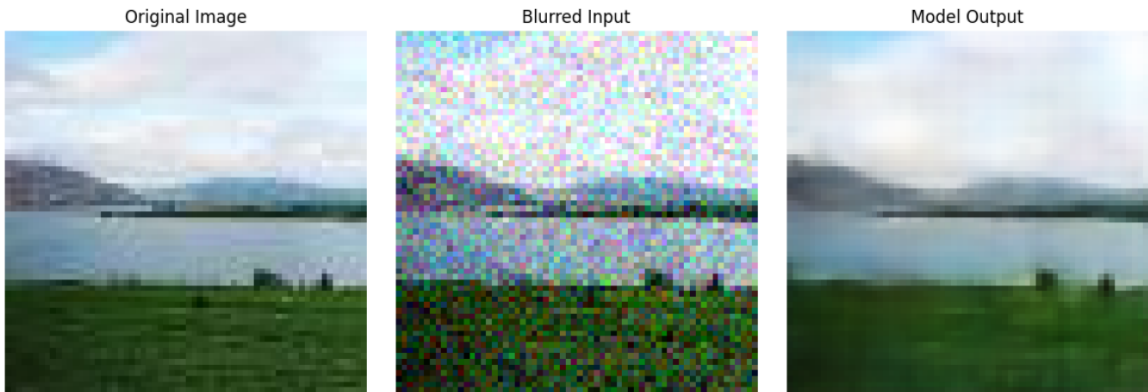


Figure 5: **Reconstruction Trio.** *Left:* Original clean image from Tiny-ImageNet-10. *Centre:* Noisy input after Gaussian noise corruption ($\mu = 0$, $\sigma = 0.1$). *Right:* Denoised output produced by the trained DAE.

Loss Function and Evaluation Metric

Why MSE is the Right Loss for Image Reconstruction

1. **Pixel-wise intensity agreement.** MSE penalises every pixel independently, directly incentivising sharp, accurate reconstructions.

2. **Smooth, well-conditioned gradients.** The gradient $\nabla_{\hat{x}} \mathcal{L}_{\text{MSE}} = \frac{2}{N}(\hat{x} - x)$ scales linearly with prediction error.
3. **Statistical consistency with Gaussian noise.** The MSE minimiser is exactly the maximum-likelihood estimator for the clean signal under Gaussian noise.
4. **Simplicity and interpretability.** Unlike perceptual losses or adversarial objectives, MSE requires no additional networks and has a direct intuitive meaning.

Why SSIM is a Better Evaluation Metric Than R^2

While R^2 is suitable for tabular regression tasks, it is a poor image quality metric for three reasons:

1. **Global, not structural.** R^2 aggregates error uniformly over all pixels without regard to structure.
2. **Insensitive to perceptual quality.** It does not account for edges, textures, or local contrast.
3. **Uninformative negative values.** R^2 can turn negative when the model underperforms a trivial mean predictor, providing no actionable insight.

SSIM is explicitly designed for images. It is sensitive to edge alignment, texture fidelity, and local contrast – the exact properties that determine perceived reconstruction quality.

Conclusion

Two experiments were conducted on Tiny-ImageNet-10 using a convolutional autoencoder. The standard reconstruction autoencoder achieved an SSIM of **71%**; the denoising variant achieved **70%** – demonstrating effective noise suppression with negligible loss in reconstruction quality.

Part 3.1: Latent Space Visualization & Analysis (2D Manifold)

Introduction

This experiment explores how an Autoencoder compresses high-dimensional data into a low-dimensional latent space. We train a symmetric MNIST Autoencoder with a 2D bottleneck ($N = 2$) independently, then visualize the latent representations of the test set to analyze how the network organizes different digit classes.

Methodology

Architecture

A symmetric undercomplete Autoencoder:

- **Encoder:** $784 \rightarrow 128$ (ReLU) $\rightarrow 64$ (ReLU) $\rightarrow 2$ (Bottleneck)
- **Decoder:** $2 \rightarrow 64$ (ReLU) $\rightarrow 128$ (ReLU) $\rightarrow 784$ (Sigmoid)

Training Details

- **Dataset:** MNIST (60,000 train / 10,000 test), pixels in $[0, 1]$
- **Optimizer:** Adam ($lr = 10^{-3}$)
- **Loss:** MSE Reconstruction Loss

- **Epochs:** 30
- **Batch Size:** 256

Results

Reconstruction Loss

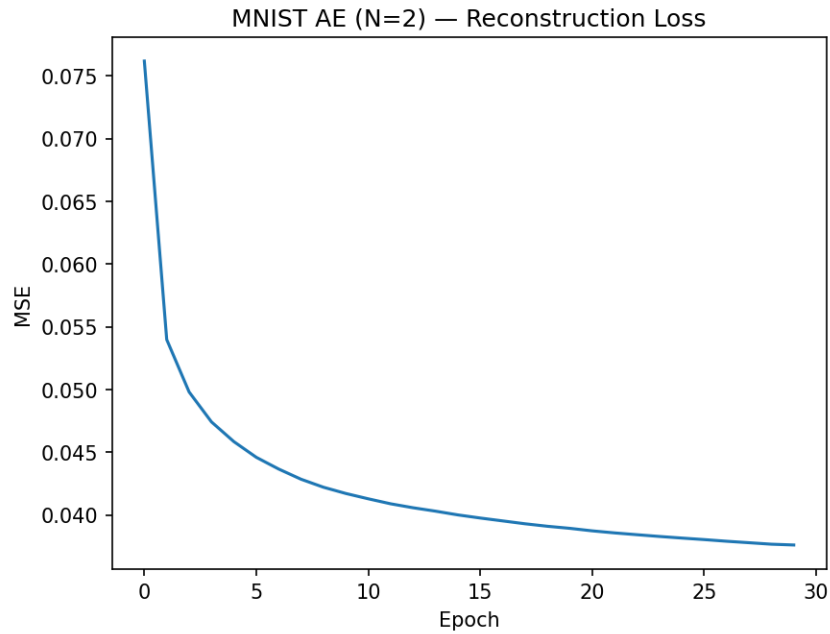


Figure 6: Reconstruction Loss (MSE) over 30 epochs for the MNIST AE with $N = 2$.

Original vs Reconstructed Images

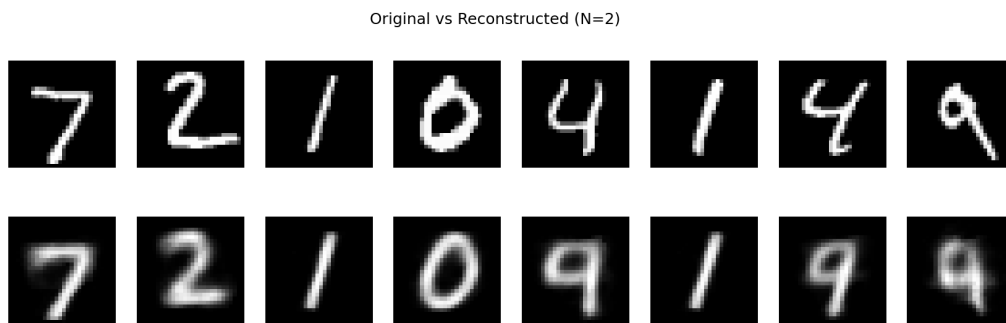


Figure 7: Original (top) vs Reconstructed (bottom) MNIST digits from the $N = 2$ Autoencoder.

2D Latent Space Scatter Plot

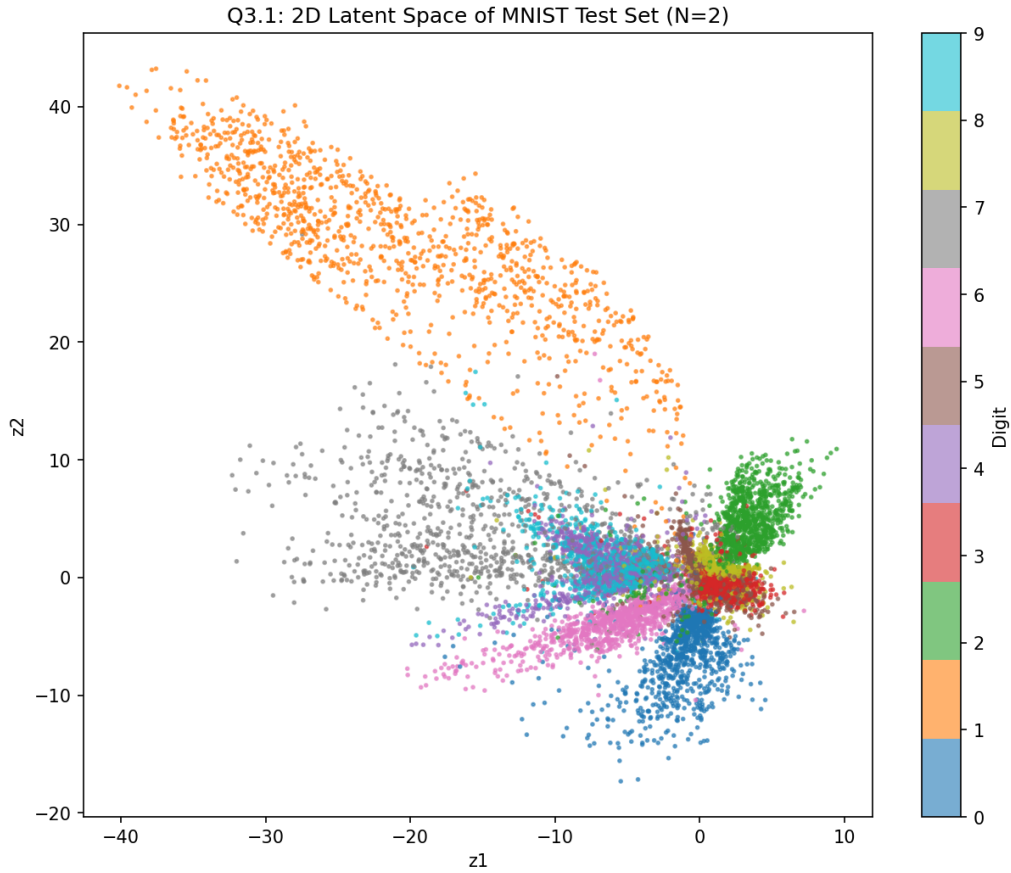


Figure 8: 2D Latent Space of MNIST test set. Each point is an image encoded to 2D, colored by its digit label (0–9).

Analysis

1. **Clustering:** Digits naturally form clusters in the 2D latent space, even though the AE was trained without any label information. This shows the network learns a meaningful representation purely from reconstruction.
2. **Similar Digits Overlap:** Visually similar digits (e.g., 4 & 9, 3 & 8, 1 & 7) tend to cluster nearby or partially overlap, because they share structural features.
3. **Distinct Digits Separate:** Structurally distinct digits (e.g., 0 and 1) are well-separated, confirming the latent space captures meaningful visual differences.

Part 3.2: Anomaly Detection using Convolutional Autoencoders

Problem Definition & Methodology

The objective of this assignment was to investigate whether Convolutional Autoencoders (CAEs) can be used for anomaly detection on Tiny-ImageNet (10 classes).

The task was to:

- Train an autoencoder on 9 classes (seen data)

- Hold out 1 class as OOD (unseen)
- Compare reconstruction error (MSE) between seen and unseen samples

Methodology:

1. Selected one class (e.g., class 3) as OOD
2. Trained a convolutional autoencoder on remaining 9 classes
3. Evaluated reconstruction error on seen and unseen (OOD) classes
4. Visualized training loss, reconstruction error per class, and original vs reconstructed images
5. Extended experiment by repeating training for each class as OOD (10 models) and computing mean and variance of reconstruction error per class

Hyperparameter Tuning & Architecture Design

Model: Convolutional Autoencoder

- Encoder: 3 Conv layers (stride=2), channels: $3 \rightarrow 32 \rightarrow 64 \rightarrow 128$
- Decoder: 3 ConvTranspose layers (mirroring encoder)
- Activation: ReLU (hidden), Tanh (output)
- No BatchNorm or Dropout

Training setup:

- Loss: Mean Squared Error (MSE)
- Optimizer: Adam ($lr = 10^{-3}$)
- Batch size: 128
- Epochs: 30

Observation: Training loss converged smoothly, indicating stable learning.

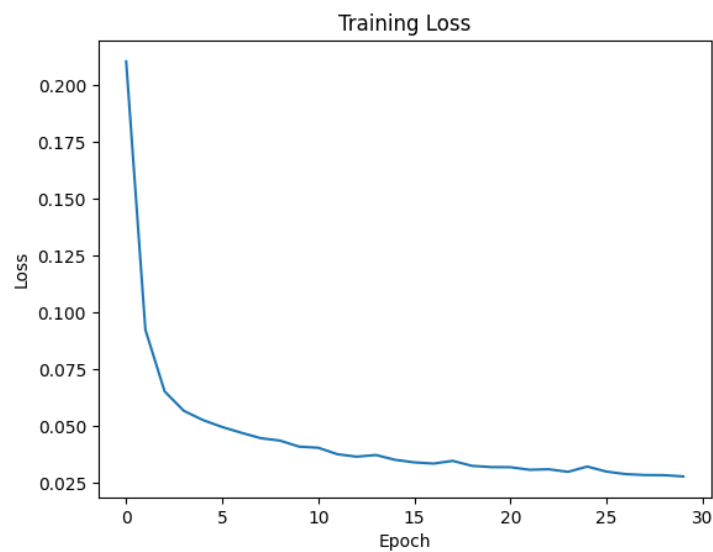


Figure 9: Training loss curve

Results, Visualizations, and Interpretations

Single OOD experiment (Class 3):

- Seen class MSE: 0.0284
- Unseen class MSE: 0.0168

Interestingly, the unseen class did not always produce higher error, indicating that reconstruction alone may not reliably detect anomalies.

Class-wise reconstruction error:

Class 0: Avg MSE = 0.028132
Class 1: Avg MSE = 0.036198
Class 2: Avg MSE = 0.018450
Class 3: Avg MSE = 0.016844
Class 4: Avg MSE = 0.028276
Class 5: Avg MSE = 0.027768
Class 6: Avg MSE = 0.043647
Class 7: Avg MSE = 0.023515
Class 8: Avg MSE = 0.020019
Class 9: Avg MSE = 0.029609



Figure 10: Reconstruction error per class (OOD highlighted)



Figure 11: Original vs reconstructed images

The reconstructions are generally good, even for the unseen class, suggesting the model learns shared visual features rather than class-specific patterns.

Key Findings and Explanations

- **Autoencoders do not reliably detect anomalies:** Reconstruction error for the unseen class is not consistently higher.
- **Error depends on feature similarity:** Classes sharing visual patterns (texture, color, structure) are reconstructed well even if unseen.
- **Low variance across OOD runs:**

Class 0: Mean=0.043255, Var=0.000002
Class 1: Mean=0.054327, Var=0.000002
Class 2: Mean=0.030177, Var=0.000001
Class 3: Mean=0.028947, Var=0.000001
Class 4: Mean=0.039932, Var=0.000001
Class 5: Mean=0.038491, Var=0.000001
Class 6: Mean=0.063730, Var=0.000005
Class 7: Mean=0.035143, Var=0.000001
Class 8: Mean=0.029017, Var=0.000001
Class 9: Mean=0.048054, Var=0.000008

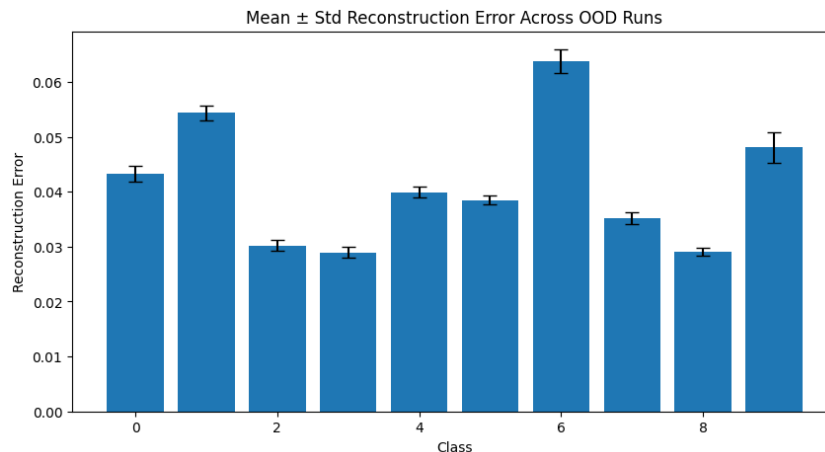


Figure 12: Mean $\pm$ Std reconstruction error across OOD runs

Conclusion:

- Reconstruction error is not strongly dependent on which class is OOD.
- CAEs learn a shared latent representation of visual features.
- This limits their effectiveness for anomaly detection in complex datasets.

Final insight: Anomaly detection using autoencoders works only when anomalies are sufficiently different in feature space, not merely unseen during training.

Part 3: Ablation Study – Effect of Output Activation Function

Objective

The objective of this ablation study is to systematically evaluate the impact of the output activation function on the performance of an Autoencoder trained on the MNIST dataset.

Specifically, we compare the reconstruction quality and convergence behavior when using **Sigmoid** and **Tanh** activation functions in the final layer.

Model Setup

A fully connected Autoencoder architecture was made using PyTorch:

- Encoder: $784 \rightarrow 128 \rightarrow 64 \rightarrow 32$ (Bottleneck)
- Decoder: $32 \rightarrow 64 \rightarrow 128 \rightarrow 784$ (Reconstructed Image)

The following training configuration was used:

- Dataset: MNIST Dataset
- Input normalization: Pixel values were scaled to $[0, 1]$
- Loss function: Mean Squared Error (MSE)
- Optimizer: Adam
- Number of epochs: 30

Two models were trained under identical conditions, differing only in the output activation function:

- **Model A:** Sigmoid activation
- **Model B:** Tanh activation

Results

Reconstruction Loss Analysis

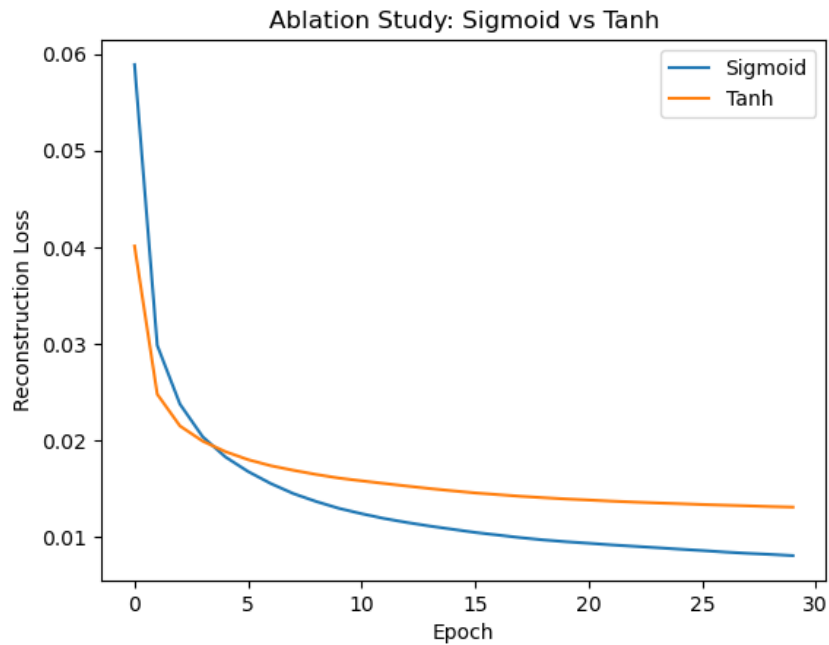


Figure 13: Reconstruction loss for Sigmoid and Tanh activations

From the loss curves:

- Both models exhibit a decreasing trend in reconstruction loss, indicating successful learning.
- The Sigmoid-based model consistently achieves lower reconstruction loss throughout training.
- The gap between the two models increases over epochs, suggesting better convergence for Sigmoid.

Reconstruction Comparison

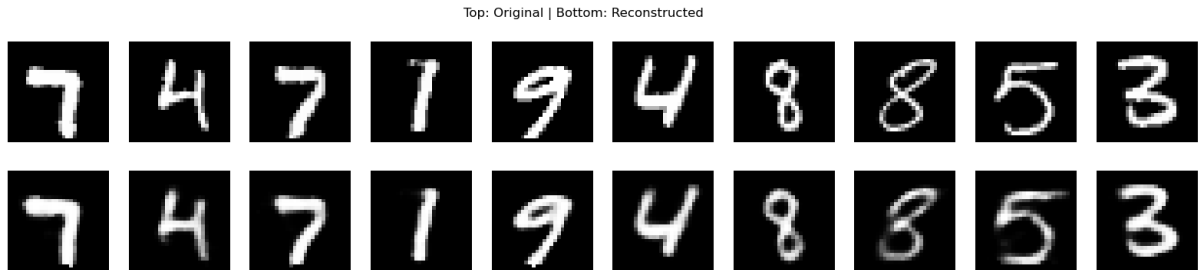


Figure 14: Reconstruction using Sigmoid activation

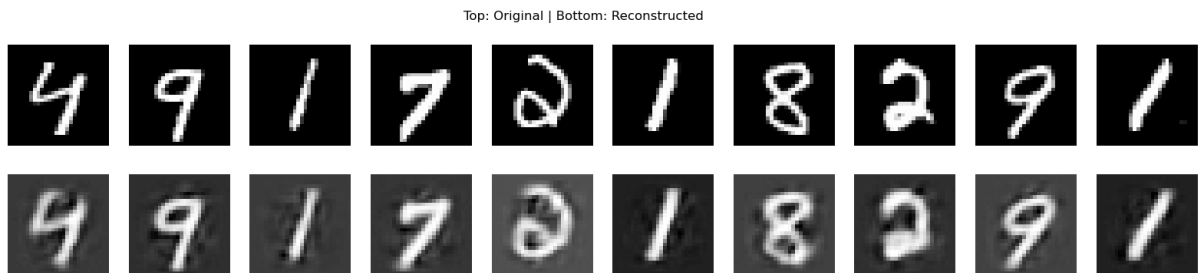


Figure 15: Reconstruction using Tanh activation

It is clearly visible that:

- The Sigmoid model produces sharper and more accurate reconstructions.
- The Tanh model exhibits noticeable blurring and distortion.
- Background regions in Tanh reconstructions appear inconsistent due to negative output values.

Discussion and Analysis

The observed performance difference can be attributed to the inherent mismatch between the output range of the activation function and the input data distribution.

The MNIST dataset is normalized such that the pixel intensities are in the range $[0, 1]$. The output ranges of the activation functions are:

- Sigmoid: $[0, 1]$
- Tanh: $[-1, 1]$

Thus, Sigmoid provides a direct mapping to the data distribution, whereas Tanh introduces negative values that do not naturally correspond to valid pixel intensities. Due to this mismatch,

the Tanh-based model is required to learn an additional transformation to align its output with the target distribution, increasing the complexity of the optimization problem.

Visual Inconsistencies: The presence of negative values in Tanh outputs leads to distortions when visualized:

- Blurred digit boundaries
- Irregular background intensities
- Loss of fine structural details

Conclusion

This ablation study demonstrates that the choice of output activation function significantly influences Autoencoder performance. When the input data is normalized to $[0, 1]$, the Sigmoid activation function is more appropriate due to its compatible output range, leading to improved reconstruction quality and faster convergence.

It is important to note that if the input data were normalized to the range $[-1, 1]$, the Tanh activation function would become a more suitable choice and could potentially produce comparable or improved performance.